COS 214 Practical 2

- Date Issued: **10 August 2026**
- Date Due: **18 August 2026** at 23:59
- Submission Procedure: **Upload an archive to FitchFork**
- Marks: **250**

1 Introduction

Objectives. In this practical you will take a broken design, make it right, then make it yours:

- diagnose a large design that misapplies design patterns, against the Gang of Four;
- redesign it correctly and **creatively**, applying the **Abstract Factory**, **State**, **Strategy**, **Composite** and **Decorator** patterns; and
- implement your design in C++11 with correct ownership and no memory leaks.

Outcomes. When you are done you should be able to recognise a misused pattern and say why it is wrong by intent (especially State vs Strategy); identify the GoF participants of each pattern; draw correct UML, object and state diagrams; and extend a pattern-based design with your own ideas without breaking it.

2 Constraints

1. Complete this assignment **in pairs**. Both partners submit; both names appear in the PDF.
2. Compile with `-std=c++11`. You may include **only** `vector`, `string`, `iostream` and `map`.
3. Your code is tested for memory leaks. Every owner frees what it owns in its destructor, and destructors up a hierarchy are `virtual`. Validate input.
4. Teaching Assistants may help, but will not give you the solution.

3 Submission Instructions

Upload one archive to FitchFork before the deadline. Zip **the files**, not the folder that contains them. No extension will be granted.

Files to submit: a `.h` and a matching `.cpp` for every class you write (you may group a pattern's classes into one header and implementation pair), your `main.cpp`, a `Makefile` that builds an executable named `wayfarer` with `-std=c++11`, and a single `PDF`. The contents required in the PDF are listed at the end of this document.

Demonstration: you will demonstrate your submission to a tutor at the marking session. Have it compiled and ready to run before your slot. Marks are deducted if your demonstration is slow to start or you cannot drive your own program quickly, so know your menus and have a short route through the features planned.

4 Mark Allocation

Task	**Marks**
Diagnosis of the broken design	40
Your redesign (UML + rationale + original ideas)	50
Movement (State)	35
Route finding (Strategy)	30
The world map (Composite)	30
Place features (Decorator)	30
Biomes and integration (Abstract Factory)	35
TOTAL	**250**

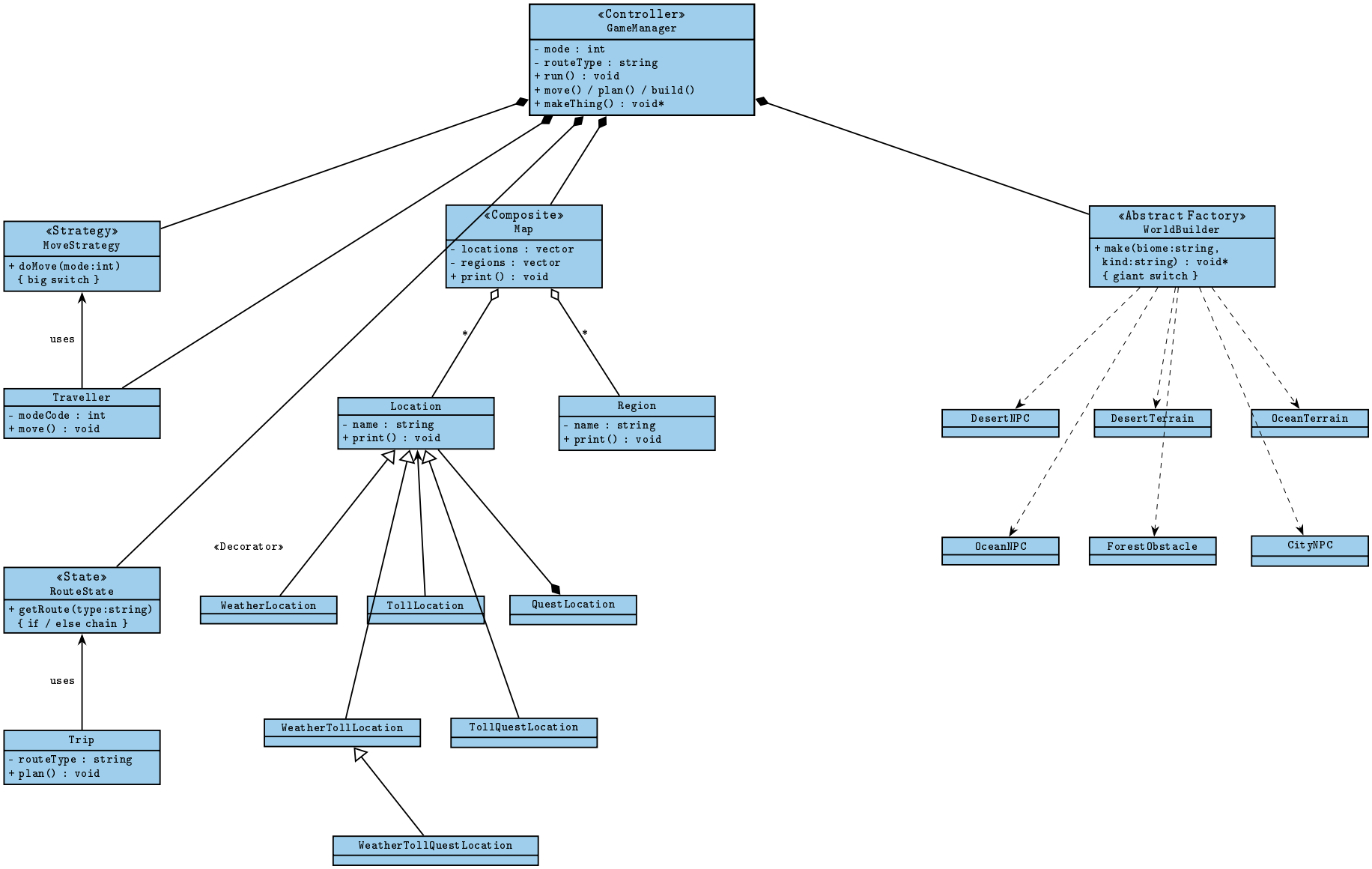
5 Scenario

Wayfarer is a world-exploration game. A traveller crosses a world map, choosing *how* to move (on foot, by bicycle, by car, by air, and whatever else you dream up), plotting a route to a destination by a chosen priority (shortest, fastest, scenic, cheapest, ...), meeting NPCs and facing obstacles that differ by *biome* (desert, ocean, forest, city, ...), on a map whose places can carry stackable features (weather, tolls, quests, ...).

A junior developer has already "designed" the engine. The result is Figure 1 on the next page: five design patterns are named, and every one of them is misused. Labels are swapped, structure is flattened, notation is wrong, one class has been made to know about everything, and nothing is ever deleted. Your job is first to diagnose this design against the Gang of Four notes, and then to throw it out and design Wayfarer properly, with your own world, your own content and your own flair.

The five patterns you must apply, and where they belong: **State** for how the traveller moves, **Strategy** for how a route is chosen, **Composite** for the world map, **Decorator** for stackable place features, and **Abstract Factory** for biome content.

Figure 1: the design you have been given (every pattern below is misused)



Legend as drawn: «...» is the developer's pattern label. No class is abstract; no class has a virtual destructor; every diamond marks an owner with no matching `delete`. Hollow triangle = inheritance, filled diamond = composition, hollow diamond = aggregation, solid arrow = association, dashed arrow = dependency.

Task 1: Diagnosis of the broken design ..... (40 marks)

Critique Figure 1 against the Gang of Four. Refer to the lecture notes for the intent, structure and participants of each pattern.

1.1 For each of the five subsystems (movement, routes, map, place features, biomes), name the pattern the design is *trying* to be and the pattern it *should* be. Exactly two labels are swapped with each other; name that pair and explain the swap in terms of *intent*. (10)

1.2 For each subsystem, list the GoF participants of the correct pattern. (10)

1.3 Identify at least **eight** distinct faults in Figure 1, drawn from all four of these categories: wrong pattern choice, wrong UML notation, broken ownership or missing virtual destructors, and excessive coupling. For each, state the rule it breaks and the runtime or maintenance consequence. (10)

1.4 Identify the class that has been made to know about everything, and explain how centralising all behaviour in one class defeats the coupling goals the patterns exist to serve. State what should own what instead. (6)

1.5 Set the patterns aside and critique the diagram purely *as a diagram*: why is it hard to read? Suggest how you would improve its layout. (4)

Task 2: Your redesign ..... (50 marks)

Throw Figure 1 out and design Wayfarer properly. This is *your* design: the world, the content and the extensions are yours to invent, as long as the five patterns are applied correctly.

2.1 Draw a corrected **UML class diagram** for the whole engine. Apply all five patterns correctly and show them clearly. Use correct notation throughout: hollow triangle for inheritance; a filled diamond where an owner must delete what it holds and a hollow diamond for a reference it does not own; a dashed «creates» arrow for instantiation; italics for abstract classes and operations; and a virtual destructor on every base class. Show the legitimate collaborations between subsystems (for example, decorators wrapping map places). (30)

2.2 Write a design rationale (about half a page): for each pattern, name your participants and justify one ownership or abstraction decision you made. (12)

2.3 Bring your own flair. Describe at least **three** original additions to the world (for example a new travel mode, biome, NPC type, place feature or route strategy). For each, name the pattern it extends and explain why it slots in without any existing class having to change. (8)

Task 3: Movement with State ..... (35 marks)

Model how the traveller moves with the **State** pattern: the Traveller is the Context, an abstract travel-mode interface is the State, and each mode is a concrete state. The same request behaves differently per mode, and the mode changes at runtime under rules you decide.

Requirements: at least three travel modes; a request (such as `move()`) that delegates to the current mode; at least two guarded transitions between modes that you design; correct participants; virtual destructors; no leaks. Be inventive with the modes and the rules that gate them.

3.1 Implement the abstract state and your concrete modes, with delegated behaviour and runtime transitions. (18)

3.2 Implement the Traveller (Context) with leak-free mode switching. (9)

3.3 Draw the **state diagram** for your modes: every state, every transition, and the guard on each transition that has one. (8)

Task 4: Route finding with Strategy ..... (30 marks)

Model how a route is chosen with the **Strategy** pattern: a Trip is the Context and an abstract route interface has several interchangeable concrete strategies, selectable at runtime.

Requirements: at least three route strategies; the strategy is swappable on the fly; correct participants; no leaks.

4.1 Implement the abstract strategy and your concrete strategies. (16)

4.2 Implement the Trip (Context), including changing strategy at runtime with correct memory management. (8)

4.3 (PDF) Using intent, explain why routes are Strategy while movement (Task 3) is State, and why the two designs are not interchangeable even though their class diagrams look alike. (6)

Task 5: The world map with Composite ..... (30 marks)

Build the world map with the **Composite** pattern: a common place interface, a region that contains other places (the composite), and a location (the leaf), forming a tree the client treats uniformly.

Requirements: a genuine tree (regions nested inside regions); the composite owns and deletes its children, so deleting the root frees the whole map; uniform traversal through the base interface.

5.1 Implement the component, the leaf and the composite, with recursive traversal and a leak-free destructor. (20)

5.2 Draw an **object diagram** of a sample map you build, with at least two levels of nesting. (10)

Task 6: Place features with Decorator ..... (30 marks)

Add stackable place features with the **Decorator** pattern. Because a decorator shares the place interface, it decorates the Composite from Task 5, so a decorated place is still a place and can sit anywhere in the map.

Requirements: at least three features that can be stacked in any combination and order; the abstract decorator owns the place it wraps and deletes it; no leaks; no subclass-per-combination. Invent your own features.

6.1 Implement the abstract decorator and your concrete decorators. (22)

6.2 Draw an **object diagram** of one place wrapped in at least two decorators, showing the wrapper chain and that exactly one place is owned at each level. (8)

Task 7: Biomes with Abstract Factory, and integration ..... (35 marks)

Build biome content with the **Abstract Factory** pattern: an abstract factory produces a *family* of related products (for example terrain, an NPC and an obstacle), with one concrete factory per biome, so a biome's products always match and can never be mixed across biomes.

Requirements: abstract product interfaces; at least two biomes, each producing a full, coherent family; families that cannot be mixed; then a `main.cpp` and `Makefile` (executable `wayfarer`, `-std=c++11`).

7.1 Implement the abstract products and at least two full biome families. (12)

7.2 Implement the abstract factory and your concrete biome factories. (11)

7.3 Write `main.cpp` and the `Makefile`. Your `main` must construct each of the five patterns, show each one working, and leak nothing. A correct, leak-free program that simply demonstrates each pattern earns up to 7 of the 12 marks. For the rest, **do more**: build a fun, menu-driven demo that lets the player choose a travel mode, pick a route strategy, walk the map, step into a biome and meet what it spawns, and stack features onto a place. Make it interactive, make it yours, and make it entertaining. Creativity and a working, playable loop are rewarded. (12)

Your PDF must contain, in this order:

1. both partners' names and student numbers;
2. Task 1: your full diagnosis of the broken design;
3. Task 2: the corrected UML class diagram, the design rationale, and your three original additions;
4. Task 3: the state diagram for your travel modes;
5. Tasks 5 and 6: the two object diagrams (your sample map, and a decorated place);
6. the written answers to every part marked (PDF).